

RADONAIX Revenue Assurance — API Reference

Plain-English reference for every backend API: what it does, what it expects, and what it returns.

Conventions (read first)

- **Base URL:** all paths are under `/api` — e.g. `http://localhost:8000/api/users`.
 - **Auth:** every endpoint except the health checks and `POST /auth/login` / `POST /auth/refresh` requires a **JWT bearer token** in the header: `Authorization: Bearer <token>`. You get the token from `POST /auth/login`. Access tokens last ~12 hours; refresh tokens ~14 days.
 - **Permissions (RBAC):** each protected endpoint needs a permission — a *feature* (e.g. `userManagement`, `pipelines`) plus an *action* (`view` to read, `edit` to change). The four roles (`admin`, `ra_lead`, `analyst`, `viewer`) each carry a matrix of these. If your role lacks the permission you get **403**.
 - **Pagination:** list endpoints accept `?limit=` (1–500, default 50) and `?offset=` (default 0).
 - **Errors:** returned as JSON `{ "error": { "code": "...", "message": "...", "details": {...} } }`. Common: **401** missing/invalid token, **403** permission denied, **404** not found, **422** invalid request body.
 - **Timestamps** are ISO-8601 strings.
-

1. Health & Dashboard (`meta`)

GET `/api/health`

- **What:** Liveness check — is the API process up.
- **Input:** none (no auth).
- **Output:** `{ status: "ok", version, environment }`.

GET `/api/health/ready`

- **What:** Readiness check — can the app serve traffic. Pings the app database and the three integrations (ClickHouse, ra-platform Postgres, Airflow). Ready only requires the app DB; integrations are optional.
- **Input:** none (no auth).
- **Output:** `{ ready: bool, checks: { app_db, clickhouse, ra_postgres, airflow } }` (each a true/false).

GET `/api/dashboard/kpis`

- **What:** Headline numbers for the dashboard, derived from the last 24h of reconciliation data (ClickHouse).
 - **Input:** none. **Permission:** `dashboard / view`.
 - **Output:** `{ assuredRevenue, matchRate, openLeakageRisk, criticalAlerts }`.
-

2. Authentication (`/auth`)

POST `/api/auth/login`

- **What:** Sign in. Validates email + password, returns an access token, a refresh token, and the user profile. Enforces account lockout after too many failures.
- **Input (body):** `{ email, password }`.
- **Output:** `{ token, refreshToken, user: { id, name, email, role, roleLabel, department, avatar, status, lastLogin, mustResetPassword } }`.

POST `/api/auth/refresh`

- **What:** Exchange a valid refresh token for a fresh access + refresh pair (rotates the refresh token; a reused/old token triggers revocation of the session).
- **Input (body):** `{ refreshToken }`.
- **Output:** `{ token, refreshToken }`.

POST /api/auth/logout

- **What:** Revoke the current session so its access token stops working immediately.
- **Input:** none (bearer token identifies the session).
- **Output:** `{ ok: true, detail }`.

POST /api/auth/change-password

- **What:** Change your own password.
- **Input (body):** `{ currentPassword, newPassword }` (new password min 8 chars).
- **Output:** `{ ok: true, detail }`.

GET /api/auth/me

- **What:** Return the signed-in user's own profile.
- **Input:** none (bearer token).
- **Output:** `AuthUser` — `{ id, name, email, role, roleLabel, department, avatar, status, lastLogin, mustResetPassword }`.

GET /api/permissions

- **What:** The catalog of permission keys (used by the role editor to know which toggles exist).
 - **Input:** none. **Permission:** `roleManagement / view`.
 - **Output:** list of `{ key, label, path }`.
-

3. User Management (/users)

GET /api/users

- **What:** List all users (paginated).
- **Input:** `?limit`, `?offset`. **Permission:** `userManagement / view`.
- **Output:** list of `UserRow` — `{ id, fullName, email, phone, department, role, status, lastLogin, createdAt }`.

POST /api/users

- **What:** Create a new user.
- **Input (body):** `{ fullName, email, password (min 8), role, phone?, department?, status?, mustResetPassword? }`. **Permission:** `userManagement / edit`.
- **Output:** the created `UserRow`.

GET /api/users/{user_id}

- **What:** Fetch one user by id. **Permission:** `userManagement / view`.
- **Output:** `UserRow`.

PATCH /api/users/{user_id}

- **What:** Update any subset of a user's fields (name, email, password, role, phone, department, status).
- **Input (body):** any of `UserUpdate` fields. **Permission:** `userManagement / edit`.
- **Output:** updated `UserRow`.

DELETE /api/users/{user_id}

- **What:** Soft-delete a user (marks them deleted; you cannot delete your own account).
 - **Input:** path id. **Permission:** `userManagement / edit`.
 - **Output:** `{ ok: true, detail }`.
-

4. Roles & Permissions (/roles)

GET /api/roles

- **What:** List all roles with their full permission matrices. **Permission:** `roleManagement / view`.

- **Output:** list of `RoleRow` — `{ id, name, description, status, permissions, createdAt, updatedAt }`.

POST `/api/roles`

- **What:** Create a role, or upsert one (pass an `id` to update, omit to create).
- **Input (body):** `{ id?, name, description?, status?, permissions? }`. **Permission:** `roleManagement / edit`.
- **Output:** `RoleRow`.

PATCH `/api/roles/{role_id}`

- **What:** Update a role's metadata only (name / description / status). Permissions go through the next endpoint.
- **Input (body):** any of `{ name, description, status }`. **Permission:** `roleManagement / edit`.
- **Output:** `RoleRow`.

PUT `/api/roles/{role_id}/permissions`

- **What:** Replace a role's entire permission matrix.
- **Input (body):** `{ permissions: { <feature>: { view, edit }, ... } }`. **Permission:** `roleManagement / edit`.
- **Output:** `RoleRow`.

5. Audit Log (`/audit-logs`)

GET `/api/audit-logs`

- **What:** List audit-trail entries (logins, user/role changes, etc.), newest first, paginated.
- **Input:** `?limit`, `?offset`. **Permission:** `settings / view`.
- **Output:** list of `AuditRow` — `{ id, actor, action, target, at }`.

6. Pipelines & Job Monitor (`/pipelines`)

GET `/api/pipelines/stages`

- **What:** Status of each processing stage (Collection → Decoding → Validation → Reconciliation → Reporting), derived from ClickHouse; falls back to representative data if upstream is unavailable.
- **Permission:** `pipelines / view`.
- **Output:** list of `{ key, name, status, duration, metric }`.

GET `/api/pipelines/kpis`

- **What:** Pipeline headline KPIs. **Permission:** `pipelines / view`.
- **Output:** `{ throughput, avgLatency, failed24h, slaBreaches }`.

GET `/api/pipelines/runs`

- **What:** Recent reconciliation runs.
- **Input:** `?limit` (1–200, default 25). **Permission:** `pipelines / view`.
- **Output:** list of `{ id, source, batch, start, end, status, records, failed }`.

GET `/api/pipelines/alerts`

- **What:** Operational alerts (from the app's `pipeline_alerts` table). **Permission:** `pipelines / view`.
- **Output:** list of `{ id, severity, stage, message, createdAt, status }`.

GET `/api/pipelines/retries`

- **What:** Jobs that are candidates for retry (surfaced from open alerts). **Permission:** `pipelines / view`.
- **Output:** list of `{ id, batch, stage, error, retryCount }`.

GET `/api/pipelines/batches` ★ (Pipeline Map)

- **What:** Live per-batch pipeline logs from the ra-platform DB, **grouped by DAG and stream**, for the last *N* hours. Currently AIR Raw + AIR Processed; SDP/MSD added later.

- **Input:** `?hours` (1–168, default 12). **Permission:** `pipelines / view`.
- **Output:** list of groups `{ dag, stream, rows: BatchLog[] }`, where each `BatchLog` has the batch id, per-stage start/end/status (watcher, decoder, ingestion, normalization...), file counts, row counts, and error/quarantine fields. Missing columns come back as `0 / ""`.

GET `/api/pipelines/batches/{batch_id}/files` ★ (Export → View files)

- **What:** Every file belonging to one batch. The source table is chosen automatically from the batch-id prefix (`AIR_PROCESSED_...` → processed file log, `AIR_RAW_...` → raw file log).
- **Input:** `batch_id` in the path. **Permission:** `pipelines / view`.
- **Output:** flat list of `FileLog` — `{ id, filename, batch_id, node_id, sequence_number, file_timestamp, file_type, file_status, integrity_flag, archived_*, watcher/picker/decoder/csv_creation/db_loading/ingestion stages, expected_record_count, actual_record_count, retry_count, last_error_step, error_message, quarantine_* }`. Returns `[]` for an unknown prefix or an empty table.

POST `/api/pipelines/jobs/{job_id}/retry`

- **What:** Trigger a retry of a job (via Airflow; queues gracefully if Airflow is disabled).
- **Input:** `job_id` in path. **Permission:** `pipelines / edit`.
- **Output:** `{ ok, detail }`.

POST `/api/pipelines/jobs/{job_id}/replay`

- **What:** Trigger a full replay of a job. **Permission:** `pipelines / edit`.
- **Output:** `{ ok, detail }`.

POST `/api/pipelines/alerts/{alert_id}/ack`

- **What:** Acknowledge an operational alert (marks it Acknowledged by the caller). **Permission:** `pipelines / edit`.
- **Output:** `{ ok, detail }`.

7. Decoders (`/decoders`)

GET `/api/decoders`

- **What:** List configured decoders. **Permission:** `settings / view`.
- **Output:** list of `{ id, name, version, status, throughput }`.

POST `/api/decoders`

- **What:** Create or update a decoder (upsert by `id`).
- **Input (body):** `{ id, name, version, status?, throughput?, config? }`. **Permission:** `settings / edit`.
- **Output:** the saved decoder row.

8. System Configuration (`/system/config`)

GET `/api/system/config`

- **What:** The singleton system config. **Permission:** `settings / view`.
- **Output:** `{ environment, retentionDays, slaMinutes, alertEmail, maintenanceMode }`.

PUT `/api/system/config`

- **What:** Update any subset of the config.
- **Input (body):** any of `{ environment, retentionDays, slaMinutes, alertEmail, maintenanceMode }`. **Permission:** `settings / edit`.
- **Output:** updated config.

9. Reports (/reports)

GET /api/reports

- **What:** List generated/queued reports (paginated). **Permission:** reports / view .
- **Output:** list of { id, name, period, status, size } .

POST /api/reports

- **What:** Request a new report. Returns immediately (HTTP 202) while it generates in the background.
- **Input (body):** { name, reportType?, period?, params? } . **Permission:** reports / edit .
- **Output:** the created ReportRow (status starts as queued/processing).

GET /api/reports/{reference}/download

- **What:** Download a completed report as a CSV file. Includes an X-Checksum-SHA256 header.
 - **Input:** reference in path. **Permission:** reports / view .
 - **Output:** the CSV file, or 404 if the report isn't finished yet.
-

10. Reconciliation (/recon) — kept live

GET /api/recon/summary

- **What:** Reconciliation summary over the last *N* hours (totals, match rate, estimated leakage), read from ClickHouse.
- **Input:** ?hours (1–720, default 24). **Permission:** dashboard / view .
- **Output:** { total, matched, amountMismatch, rawOnly, procOnly, matchRate, estimatedLeakage } .

GET /api/recon/records

- **What:** Individual reconciliation records, optionally filtered by status, paginated.
 - **Input:** ?status, ?limit, ?offset . **Permission:** workbench / view .
 - **Output:** list of { recordType, txnId, nodeId, subscriberNum, rawAmount, procAmount, rawBalance, procBalance, status, createTime } .
-

11. Case Management (/cases) — hidden in the first patch

These endpoints exist in code but are **not mounted** in the first application patch (the Case Management UI is hidden). Listed for completeness.

GET /api/cases

- **What:** List cases (paginated, filter by status). **Permission:** caseManagement / view .
- **Output:** list of { id, reference, title, severity, status, owner, updated, estimatedImpact } .

POST /api/cases

- **What:** Open a new case.
- **Input (body):** { title, description?, severity?, status?, owner?, linkedTxnId?, estimatedImpact? } . **Permission:** caseManagement / edit .
- **Output:** CaseRow .

GET /api/cases/{case_id}

- **What:** Full case detail including comments. **Permission:** caseManagement / view .
- **Output:** CaseRow + { description, linkedTxnId, comments[] } .

PATCH /api/cases/{case_id}

- **What:** Update a case (title, description, severity, status, owner, impact). **Permission:** caseManagement / edit .

- **Output:** `CaseRow`.

POST `/api/cases/{case_id}/comments`

- **What:** Add a comment to a case.
- **Input (body):** `{ body }`. **Permission:** `caseManagement / edit`.
- **Output:** the created comment `{ id, author, body, createdAt }`.

12. Assurance Workbench (`/workbench`) — 🚫 hidden in the first patch

Not mounted in the first application patch (Workbench UI hidden). Listed for completeness.

GET `/api/workbench/queries`

- **What:** List saved investigation queries. **Permission:** `workbench / view`.
- **Output:** list of `{ id, reference, name, owner, count }`.

POST `/api/workbench/queries`

- **What:** Save a new query.
- **Input (body):** `{ name, definition? }`. **Permission:** `workbench / edit`.
- **Output:** `SavedQueryRow`.

GET `/api/workbench/stats`

- **What:** Workbench summary stats. **Permission:** `workbench / view`.
- **Output:** `{ openInvestigations, closedThisWeek, avgResolutionDays }`.

Quick index

Area	Endpoints
Health/Dashboard	<code>GET /health</code> , <code>GET /health/ready</code> , <code>GET /dashboard/kpis</code>
Auth	<code>POST /auth/login</code> , <code>/auth/refresh</code> , <code>/auth/logout</code> , <code>/auth/change-password</code> , <code>GET /auth/me</code> , <code>GET /permissions</code>
Users	<code>GET/POST /users</code> , <code>GET/PATCH/DELETE /users/{id}</code>
Roles	<code>GET/POST /roles</code> , <code>PATCH /roles/{id}</code> , <code>PUT /roles/{id}/permissions</code>
Audit	<code>GET /audit-logs</code>
Pipelines	<code>GET /pipelines/{stages,kpis,runs,alerts,retries,batches}</code> , <code>GET /pipelines/batches/{id}/files</code> , <code>POST /pipelines/jobs/{id}/{retry,replay}</code> , <code>POST /pipelines/alerts/{id}/ack</code>
Decoders	<code>GET/POST /decoders</code>
System	<code>GET/PUT /system/config</code>
Reports	<code>GET/POST /reports</code> , <code>GET /reports/{ref}/download</code>
Reconciliation	<code>GET /recon/{summary,records}</code>
Cases 🚫	<code>GET/POST /cases</code> , <code>GET/PATCH /cases/{id}</code> , <code>POST /cases/{id}/comments</code>
Workbench 🚫	<code>GET/POST /workbench/queries</code> , <code>GET /workbench/stats</code>

★ = built this engagement (Pipeline Map + Export). 🚫 = hidden in the first application patch.